

FixIT

A Retro Computer Repair Shop

Game Development Project Report

Author: Yared Abera

Student ID: [fill in]

Course: Game Development Project - University of Florence (UniFi)

Instructor: Prof. Francesco Tiezzi

Engine: Unity 6 (6000.4.10f1), C#, URP 2D

Date: 8 June 2026

Source: github.com/theHabesa1/FixIT

macOS build: github.com/theHabesa1/FixIT/releases/tag/osx-1.0

1. Concept

FixIT is a 2D pixel-art game in which the player runs a computer repair shop. Customers walk in with broken devices or shopping orders. Repairs are resolved by solving short computer-science puzzles, and sales are resolved by fetching the correctly specified hardware from a stockroom. The player earns money and knowledge, and the game tracks the player's mastery of each CS topic over time.

2. Core Gameplay Loop

- Move around the shop (WASD / arrow keys) and approach a waiting customer.
- Press E (or click) to serve them.
- Repair customers launch a topic mini-game; buyers send you to the back store.
- Success grants Knowledge Points and Money; failure costs an attempt or points.
- Difficulty and rewards scale with the player's mastery of that topic.

3. Peculiar Feature - The Knowledge System (FSM)

Each CS topic (Logic, Binary, Circuits, Memory) is modelled as a finite-state machine with three states: Novice, Intermediate and Expert. Transitions are deterministic and driven by performance:

- Three correct results in a row -> promotion to the next level.
- Two wrong results in a row -> demotion to the previous level.

The current state selects the puzzle-difficulty tier and the reward tier (Knowledge {12, 24, 40}, Money {15, 30, 55} for Novice/Intermediate/Expert). A mastery board on the shop wall shows each topic's stars and level live. This adaptive-difficulty FSM is the project's distinctive mechanic.

4. The Mini-Games

- Logic Gates: toggle the input switches so the live circuit output matches the target. Wires and the output LED glow as the signal propagates.
- Binary Decoder: a corrupted terminal shows a binary (or hex) value; type its decimal value on the keypad before running out of lives.
- Circuit Tracer: click adjacent nodes to route power from the battery to the bulb, around blocked components, before the power bar drains.
- RAM Matcher: install RAM modules into dual-channel DIMM slots so the channels mirror by type, size and speed.

Each generator reads the topic's mastery level to pick board size, number of gates/bits, obstacle count, etc.

5. The Sales Loop (Back Store)

Some customers are buyers. Taking an order displays the requested product and exact specification and stocks the back-store shelves so exactly one item matches. The player walks through a door into a second room, clicks the matching shelf to pick the item up, returns, and sells it: a correct spec earns the sale price, a wrong spec costs knowledge points. The buyer waits 30-45 seconds.

6. Architecture

- Code-driven: every scene is built at runtime from a single bootstrap component; no prefabs or manual Inspector wiring.
- GameManager auto-bootstraps before the first scene via `RuntimeInitializeOnLoadMethod`, so any scene runs standalone.
- UIFactory builds all UI (canvas, panels, buttons, text) in code.
- Pixels generates all sprites (characters, furniture, RAM) from char-grid maps; Sfx synthesises all sound effects at runtime - no external art or audio assets.
- A shared KnowledgeSystem, SceneLoader and Theme palette tie the scenes together.

7. Visual and Audio Design

The look follows a fixed retro palette (deep navy background, neon green, purple and amber accents) with two pixel fonts (Press Start 2P, VT323). A full-screen CRT overlay adds scanlines and a vignette. Characters and furniture are hand-authored pixel sprites generated in code. All sound is procedurally generated (clicks, correct/wrong tones).

8. Build and Run

- macOS executable: download FixIT-macOS.zip from the GitHub release (osx-1.0), unzip, and open FixIT.app.
- First launch on macOS: right-click the app -> Open -> Open (the build is unsigned, so Gatekeeper prompts once).
- Controls: WASD / arrows move; E interact; R return to shop; Enter or click on the title screen.
- A GitHub Actions workflow (GameCI) is also included to build macOS in CI.

9. Conclusion and Possible Extensions

FixIT delivers a complete loop - explore, serve customers, solve puzzles, sell hardware, and progress through an adaptive mastery system - built entirely in code. Future work could add more CS topics, a day/earnings target, shop upgrades, and persistent save data.